

Reviewer Pack — Minimal p-Adic L-Function Rank Correlation with Communicatio...

Entry 70a6d794a81e · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-01 03:48:43 UTC

Minimal p-Adic L-Function Rank Correlation with Communication Rank Growth

Entry ID: 70a6d794a81e **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-01 03:48:43 UTC

1. Conjecture

Field A (mathematical branch): p-Adic Analysis **Field B** (complexity object): Communication Complexity

Statement:

For every k -colorable graph G with n vertices, the minimal rank of its associated p-adic L-function ($\text{lrank}(G)$) is linearly correlated with the communication rank growth rate ($\text{crg_rate}(G)$), such that $\text{lrank}(G) = \Theta(\text{crg_rate}(G))$.

Rationale (proposer's reasoning):

p-Adic analysis provides a rich framework for studying arithmetic properties, while communication complexity measures the amount of information exchanged between parties. The conjecture suggests that these two seemingly unrelated fields may be connected through their influence on the structure of graphs.

Taxonomy category: p-adic_analysis_communication_complexity_correlation (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): d9937490fd43c212

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For every k -colorable graph G with n vertices, if the Pearson correlation coefficient between $\text{lrank}(G)$ and $\text{crg_rate}(G)$ is ≥ 0.8 for all graphs and the mean of $\text{lrank}(G)$ values across seeds is ≤ 3 , this supports the conjecture.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: ADJACENT_OK against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - p-adic L-function AND communication complexity - graph p-adic L-function rank AND communication rank growth - linear correlation between $\text{lrnk}(G)$ and $\text{crg_rate}(G)$ in p-adic analysis

Top relevant hits considered: - [http://arxiv.org/abs/1309.3982v3] Desingularization of complex multiple zeta-functions, fundamentals of p -adic multiple L -functions, and evaluation of - [http://arxiv.org/abs/math-ph/0512018v2] On Phase Transitions for P -Adic Potts Model with Competing Interactions on a Cayley Tree - [http://arxiv.org/abs/2408.00810v3] p-adic Equiangular Lines and p-adic van Lint-Seidel Relative Bound - [http://arxiv.org/abs/1601.00010v1] On pairs of p-adic L-functions for weight two modular forms - [http://arxiv.org/abs/2407.06983v4] On the p -adic L -function and Iwasawa Main Conjecture for an Artin motive over a CM field - [http://arxiv.org/abs/1901.01957v6] (p -adic) L -functions and (p -adic) (multiple) zeta values - [http://arxiv.org/abs/1808.08676v3] Constraining the p-mode-g-mode tidal instability with GW170817 - [http://arxiv.org/abs/0901.0512v4] Expected Performance of the ATLAS Experiment - Detector, Trigger and Physics

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def generate_k_colorable_graph(n, k):
    if n <= 0 or k <= 1:
        return None

    graph = [[None for _ in range(n)] for _ in range(n)]
    available_colors = set(range(1, k + 1))

    def is_valid_color(i, j, color):
        for x in range(n):
            if graph[i][x] == color or (graph[x][i] is not None and graph[x][i] == color):
                return False
        return True

    def backtrack(i):
        if i == n:
            return True

        available_colors.discard(graph[i-1][i-1])
```

```

        for color in available_colors:
            if is_valid_color(i, i-1, color):
                graph[i][i-1] = color
                graph[i-1][i] = color
                if backtrack(i + 1):
                    return True

        available_colors.add(graph[i-1][i-1])
        return False

    if not backtrack(0):
        return None

    return graph

def compute_p_adic_l_function_rank(graph):
    n = len(graph)
    lrank = 0
    for i in range(n):
        for j in range(i + 1, n):
            if graph[i][j] is not None:
                lrank += 1
    return lrank

def compute_communication_rank_growth_rate(graph):
    n = len(graph)
    communication_rank = [0] * n
    for i in range(n):
        for j in range(i + 1, n):
            if graph[i][j] is not None:
                communication_rank[i] += 1
                communication_rank[j] += 1

    max_communication_rank = max(communication_rank)
    return max_communication_rank / (n * (n - 1) // 2)

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n_max = 40
    k = 3
    instances_tested = 0
    lrank_values = []
    crg_rate_values = []

    for _ in range(30):
        graph = generate_k_colorable_graph(n_max, k)
        if graph is None:
            continue

        lrank = compute_p_adic_l_function_rank(graph)
        crg_rate = compute_communication_rank_growth_rate(graph)

        lrank_values.append(lrank)
        crg_rate_values.append(crg_rate)

```

```

    instances_tested += 1

if not lrank_values or not crg_rate_values:
    return {
        "metric_name": "lrank vs crg_rate",
        "metric_value": None,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": False,
        "counterexample": "graph_generation_failed"
    }

mean_lrank = sum(lrank_values) / len(lrank_values)
std_lrank = math.sqrt(sum((x - mean_lrank) ** 2 for x in lrank_values) / len(lrank_values))
correlation_coefficient = sum((lrank_values[i] - mean_lrank) * (crg_rate_values[i] - crg_rate_mean)) / (len(lrank_values) * std_lrank * std_crg_rate)

return {
    "metric_name": "lrank vs crg_rate",
    "metric_value": correlation_coefficient,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": correlation_coefficient >= 0.8 and mean_lrank <= 3,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] if sys.argv[1:] else [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(x["metric_value"] for x in results if x["metric_value"] is not None) / len(results)
    std_metric_value = math.sqrt(sum((x["metric_value"] - mean_metric_value) ** 2 for x in results if x["metric_value"] is not None) / len(results))
    support_fraction = sum(1 for x in results if x["conjecture_holds"]) / len(results)

    if all(x["conjecture_holds"] for x in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(x["seed"] for x in results if not x["conjecture_holds"])
        counterexample = "lrank vs crg_rate"
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

```

Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_65c01def.py", line 123, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_65c01def.py", line 106, in run_trial
    correlation_coefficient = sum((lrank_values[i] - mean_lrank) * (crg_rate_values[i] -
crg_rate_values[0]) for i in range(len(lrank_values))) / (len(lrank_values) * std_lrank * math.sqrt(s
crg_rate_values[0]) ** 2 for x in crg_rate_values)))
    ~~~~~
ZeroDivisionError: float division by zero

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from evaluating the Pearson correlation coefficient and mean rank values required to support or falsify the conjecture. | next: Investigate the cause of the crash in the test code and attempt to run the test again to gather the necessary data for further analysis.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	18547
2	propose	ollama_remote	glm4:latest	0	0	23996
3	preregistration	ollama_remote	glm4:latest	0	0	9380
4	novelty	ollama_remote	glm4:latest	0	0	13754
5	novelty	ollama_remote	glm4:latest	0	0	14373
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15100
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13227
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19797
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14906
10	judge	ollama_remote	glm4:latest	0	0	11538

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 154620 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in [research/audit/70a6d794a81e.jsonl](#))

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice

3. The full LLM transcripts are in `research/audit/70a6d794a81e.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/70a6d794a81e.tar.gz` (if generated)